

AMS — Tech Stack

Phase 1 | Open-Source | March 2026

1. Frontend

. Frontend — React 19 + TypeScript (react.dev | typescriptlang.org)

✓ Why we chose it

- React 19 Compiler reduces unnecessary re-renders by 30–60% with huge community support
- TypeScript adds static typing — catches bugs at compile time, not in production
- Type-safe props, API responses, and component contracts — critical for a multi-role system
- Vite for build tooling — blazing fast dev server and production builds (replaces CRA)
- React Query for server state — clean async data fetching, caching, and background refresh
- Component-based architecture — reusable UI blocks (AssetCard, QRModal, RoleGuard)
- Free to host on Vercel (auto-deploy from GitHub)

⚠ Know the limits

- TypeScript adds initial setup time — but pays off immediately in a team environment
- React is a library, not a framework — team must decide folder structure & state management conventions
- No SSR by default (unlike Next.js) — but not needed for an authenticated admin/employee portal

2. Backend

. Backend — FastAPI + Pydantic (Python 3.12) (fastapi.tiangolo.com)

✓ Why we chose it

- Pydantic models provide end-to-end type safety from DB to API to frontend
- Shared schema definitions between API models and DB models — no desync between layers
- FastAPI is high-performance and async-first — handles concurrent API requests efficiently
- Auto-generated OpenAPI docs (/docs) — zero extra effort, always up to date
- Pydantic v2 validates all request/response bodies at runtime — catches bad data instantly
- Rich ecosystem: python-jose (JWT), passlib (bcrypt), qrcode, sqlalchemy — all free packages
- Easy to containerise with Docker for consistent dev-to-prod deploys
- Free hosting on Render.com free tier

⚠ Know the limits

- Python needs a build/container step — slightly more CI setup than a simple JS server
- FastAPI has no built-in structure — enforce conventions (routers / services / schemas) from day one

3. Authentication — Centralised Auth Service with RBAC

. Auth — Common Auth Service (JWT + bcryptjs + RBAC) ([jsonwebtoken v9](https://jwt.io) + [bcryptjs v2](https://npmjs.com/package/bcryptjs) | jwt.io)

✓ Why we chose it

- Single dedicated Auth Service shared across all AMS modules — one source of truth for identity
- RBAC defined centrally: Admin and Employee roles with permission maps — enforced at API Gateway
- JWT is stateless — server issues signed token, no session store needed, scales horizontally
- Access token (8hr) + Refresh token (7 days) pattern: secure short-lived access with seamless UX
- bcryptjs hashes passwords with salt rounds — industry standard, free npm package
- HTTP-only cookie for refresh token — prevents XSS from stealing refresh credentials
- Auth Service is built once, reused if more services or modules are added in Stage 2
- Designed to plug in SSO (Google / Microsoft OAuth2) in Stage 2 with minimal change

⚠ Know the limits

- Stateless JWT: can't instantly revoke a token mid-session — must wait up to 8hr for expiry
- Logout requires a token denylist (in-memory or DB table) to invalidate before expiry — implement from day one
- Team must handle token refresh flow on the frontend carefully to avoid silent auth gaps
- Auth Service becomes a single point of failure — add health check + uptime monitoring

4. Database

. Database — PostgreSQL via Supabase (PostgreSQL 15) (supabase.com)

✅ Why we chose it

- PostgreSQL is battle-tested, ACID-compliant relational DB — ideal for structured asset + assignment data
- Supabase gives free hosted PostgreSQL: 500MB DB + 1GB file storage + daily backups
- TypeScript-friendly: use Prisma ORM for type-safe DB queries — generated types from schema
- Built-in Supabase dashboard to browse and edit data without a separate DB tool
- Row-level security (RLS) can enforce employee-sees-own-data rules at DB layer
- Real-time subscriptions available (useful for Stage 2 live admin dashboard)

⚠ Know the limits

- Free project pauses after 1 week of inactivity — set up a keep-alive cron ping
- 500MB limit is enough for ~100,000 assets — plan upgrade path when growing
- Supabase is a managed service — if it has downtime, your DB is impacted

5. QR Code Generation

. QR Generation — qrcode (npm) (v1.5+ | npmjs.com/package/qrcode)

✅ Why we chose it

- Generates QR codes entirely server-side — no external API, no cost, no rate limit
- TypeScript types available (@types/qrcode) — fully typed integration
- Outputs PNG, SVG, or base64 — flexible for file download, email embed, and print
- Encodes asset deep-link URL: scanning opens authenticated portal to asset detail
- Single function call: QRCode.toFile() or QRCode.toDataURL() — minimal integration effort
- Sub-1 second generation — meets the <1s performance requirement

⚠ Know the limits

- QR codes are static — if the deep-link URL structure changes, old QRs break (version URLs carefully)
- No built-in logo/branding — plain black & white QR (customisable with canvas in Stage 2)
- Damaged sticker = unreadable QR — always print Asset ID as text alongside the QR label

6. Telemetry & Observability

. Telemetry — Loguru + GlitchTip + Uptime Kuma (github.com/Delgan/loguru | glitchtip.com | uptime.kuma.pet)

✅ Why we chose it

- Loguru: structured JSON logging for every API request — timestamp, user_id, action, duration, status · lightweight Python-native logger
- Custom middleware logs API response times — flag any endpoint > 500ms automatically
- Business event logging: QR generated, asset assigned, login failed — queryable audit trail
- Uptime Kuma free: monitors /health endpoint every 5 minutes — alerts via email on downtime · free self-hosted dashboard
- GlitchTip SDK tracks server + client errors, QR scan failures — free open-source Sentry alternative

⚠ Know the limits

- GlitchTip free tier: 1K events/month — sufficient for Phase 1 scale
- Loguru logs to file/console only in Phase 1 — no log aggregation UI (upgrade to Logtail/Datadog in Stage 2)
- Uptime Kuma free: 5-minute check intervals — may miss sub-5min outages
- OpenTelemetry + distributed tracing deferred to Stage 2 — too heavy for Phase 1

7. Hosting & Deployment

. Hosting & CI/CD — Vercel + Render + GitHub Actions + Docker (vercel.com)

✓ Why we chose it

- Vercel: free frontend hosting for React — auto-deploys on every GitHub push, zero config
- Render.com: free backend hosting — Docker support, free TLS, managed environment variables
- GitHub Actions: free CI/CD — lint, type-check, test, build, and deploy on every PR
- Docker ensures identical environment from local → staging → production — no 'works on my machine'
- TypeScript build step integrated into GitHub Actions pipeline automatically
- Preview deployments on Vercel for every PR — review UI changes before merging

⚠ Know the limits

- Render free tier spins down after 15min idle — upgrade to Starter (\$7/mo) to eliminate cold starts for production
- Vercel free: 100GB bandwidth/month — monitor usage as employee count grows
- No persistent disk on Render free — never write files locally on server (use Supabase Storage)
- GitHub Actions: 2,000 free minutes/month on private repos — sufficient for Phase 1

Quick Reference Summary

Layer	Tool	Cost	Key Reason
Frontend	React 19 + TypeScript	Free	Type-safe UI, Vite build, Vercel deploy, component reuse
Backend	FastAPI + Pydantic	Free	Shared types with frontend, fast API, free on Render
Auth	Auth Service (JWT + RBAC)	Free	Centralised identity, role enforcement at gateway, SSO-ready
Database	PostgreSQL / Supabase	Free 500MB	Relational, type-safe via Prisma ORM, managed backups
QR Code	qrcode (npm)	Free	Server-side, typed, PNG/SVG, <1s generation
Telemetry	Loguru + GlitchTip + Uptime Kuma	Free	Structured logs, error capture, uptime alerts

Storage	Supabase Storage	Free 1GB	S3-compatible CDN, same platform as DB
CI/CD	GitHub Actions + Docker	Free	Auto type-check, build & deploy on every push